Scenario Testing Report

Sign In and Sign Up Features - sprint5-with-bugs

1. Executive Summary

This report presents the comprehensive scenario testing analysis for the sign in and sign up features of the sprint5-with-bugs application. The testing approach covers 25 detailed scenarios including happy path testing, negative testing, security testing, boundary testing, and data quality testing. A custom data generation tool has been developed to support these scenarios with 500+ test data rows.

2. Project Overview

Application: sprint5-with-bugs
Features Tested: Sign In (Login) and Sign Up (Registration)
Technology Stack:

- Frontend: Angular (UI/src/app/auth/)
- Backend: PHP Laravel (API/app/Http/Controllers/)
- Testing Tool: Custom Python Data Generator

2.1 Team Member Feature Assignment

Member	Student ID	Features Assigned
Nguyen Quoc Khanh	22127188	Contact Form and Contact Upload File
Truong Bao Kiet	22127223	Login and Sign up
Ho Tien Phat	22127321	Tool Architecture & OOP Implementation, Batch/Shell Script Integration, Documentation
Nguyen Quang Khai	22127477	Cart and Checkout

3. Data Fields and Their Randomized Ranges/Rules

3.1 Sign Up Form Fields (11 fields)

Field	Type	Validation Rules	Randomized Ranges/Rules	Test Coverage

Field	Type	Validation Rules	Randomized Ranges/Rules	Test Coverage
first_name	String	Required + Letters only	40+ names: John, Jane, Michael, Sarah, David, Emily, Robert, Lisa, James, Jennifer, William, Jessica, Richard, Amanda, Thomas, Nicole, Christopher, Stephanie, Daniel, Melissa, Matthew, Ashley, Anthony, Elizabeth, Mark, Megan, Donald, Lauren, Steven, Rachel, Paul, Kayla, Andrew, Amber, Joshua, Brittany, Kenneth, Danielle, Kevin, Rebecca, Brian, Michelle	59 test cases
last_name	String	Required + Letters only	40+ surnames: Smith, Johnson, Williams, Brown, Jones, Garcia, Miller, Davis, Rodriguez, Martinez, Hernandez, Lopez, Gonzalez, Wilson, Anderson, Thomas, Taylor, Moore, Jackson, Martin, Lee, Perez, Thompson, White, Harris, Sanchez, Clark, Ramirez, Lewis, Robinson, Walker, Young, Allen, King, Wright, Scott, Torres, Nguyen, Hill, Flores, Green	49 test cases
email	String	Required + Email format	10 domains: gmail.com, yahoo.com, hotmail.com, outlook.com, aol.com, icloud.com, protonmail.com, mail.com, live.com, msn.com Format: firstname.lastname{number}@domain	41 test cases
password	String	Required + 9-40 chars	Length: 9-40 characters Characters: a-z, A-Z, 0-9, !@#$%^&* Random generation with mixed character types	51 test cases
phone	String	Required + Numbers only	Range: 1000000000-9999999999 Format: 10-digit numbers only No special characters or formatting	38 test cases
dob	String	Required	Date range: 1950-01-01 to 2005-12-31 Format: YYYY-MM-DD Realistic age distribution	47 test cases
address	String	Required	Street numbers: 1-9999 Street types: Main St, Oak Ave, Pine Rd, Elm St, Maple Dr, Cedar Ln Format: {number} {street_type}	42 test cases
city	String	Required	29 major US cities: New York, Los Angeles, Chicago, Houston, Phoenix, Philadelphia, San Antonio, San Diego, Dallas, San Jose, Austin, Jacksonville, Fort Worth, Columbus, Charlotte, San Francisco, Indianapolis, Seattle, Denver, Washington, Boston, El Paso, Nashville, Detroit, Oklahoma City, Portland, Las Vegas, Memphis, Louisville	52 test cases

Field	Type	Validation Rules	Randomized Ranges/Rules	Test Coverage
state	String	Required	50 US state abbreviations: AL, AK, AZ, AR, CA, CO, CT, DE, FL, GA, HI, ID, IL, IN, IA, KS, KY, LA, ME, MD, MA, MI, MN, MS, MO, MT, NE, NV, NH, NJ, NM, NY, NC, ND, OH, OK, OR, PA, RI, SC, SD, TN, TX, UT, VT, VA, WA, WV, WI, WY	53 test cases
country	String	Required	18 countries: United States, Canada, United Kingdom, Germany, France, Australia, Japan, Italy, Spain, Brazil, Mexico, India, China, South Korea, Netherlands, Sweden, Switzerland, Norway	44 test cases
postcode	String	Required	Range: 10000-99999 Format: 5-digit numbers US ZIP code format	49 test cases

3.2 Sign In Form Fields (2 fields)

Field	Type	Validation Rules	Randomized Ranges/Rules	Test Coverage
email	String	Required + Email format	Same as sign up email generation Realistic email patterns with random numbers	41 test cases
password	String	Required + 9-40 chars	Same as sign up password generation Random length and character mix	51 test cases

4. Screenshots of the Tool in Use

4.1 Tool Execution Screenshot

```
Data Generation Tool for Sign In and Sign Up Features
=====================================================

Running Python data generator...
Generating 500 test data rows for Sign In and Sign Up features...
Successfully generated 525 test data rows!
File saved as: 500_test_data_rows.csv

=== SUMMARY ===
Total rows generated: 525
Unique scenarios: 16
Unique fields: 11

=== SCENARIO DISTRIBUTION ===
Boundary Value: 25 rows (4.8%)
Duplicate Email: 10 rows (1.9%)
```

```
Empty Field: 75 rows (14.3%)
Invalid Format: 75 rows (14.3%)
Leading/Trailing Spaces: 15 rows (2.9%)
Null Values: 15 rows (2.9%)
Password Too Long: 25 rows (4.8%)
Password Too Short: 25 rows (4.8%)
SQL Injection: 15 rows (2.9%)
Special Characters: 25 rows (4.8%)
Unicode Characters: 25 rows (4.8%)
Valid Sign In: 50 rows (9.5%)
Valid Sign Up: 100 rows (19.0%)
Very Long Input: 15 rows (2.9%)
Whitespace Only: 15 rows (2.9%)
XSS Attack: 15 rows (2.9%)

=== FIELD DISTRIBUTION ===
address: 42 rows (8.0%)
city: 52 rows (9.9%)
country: 44 rows (8.4%)
dob: 47 rows (9.0%)
email: 41 rows (7.8%)
first_name: 59 rows (11.2%)
last_name: 49 rows (9.3%)
password: 51 rows (9.7%)
phone: 38 rows (7.2%)
postcode: 49 rows (9.3%)
state: 53 rows (10.1%)

Press Enter to exit...
```

4.2 Generated CSV File Structure

```
Test Data ID,Scenario,Field Name,Test Data,Data Type,Validation
Rule,Expected Result,Notes
TD001,Valid Sign Up,state,IA,String,Required,Valid,Standard valid input
TD002,Valid Sign Up,postcode,20851,String,Required,Valid,Standard valid
input
TD003,Valid Sign Up,phone,4899622610,String,Required + Numbers
only,Valid,Standard valid input
TD004,Valid Sign Up,phone,1682318383,String,Required + Numbers
only,Valid,Standard valid input
TD005,Valid Sign Up,city,Seattle,String,Required,Valid,Standard valid
input
...
```

5. Explanation of Process and Steps

5.1 Overall Testing Process

1. **Requirements Analysis**
    - Analyze sign in and sign up features
    - Identify all input fields and validation rules
    - Determine testing scenarios and priorities
2. **Data Generation Tool Development**
    - Design tool architecture using Python
    - Implement data pools for realistic test data
    - Create algorithms for valid and invalid data generation
3. **Scenario Definition**
    - Define 25 comprehensive test scenarios
    - Categorize scenarios by priority and type
    - Map scenarios to specific test data requirements
4. **Test Data Generation**
    - Execute data generation tool
    - Generate 525 test data rows
    - Validate data quality and distribution
5. **Scenario Execution Planning**
    - Create test execution plan
    - Define test steps for each scenario
    - Prepare expected results and validation criteria

5.2 Detailed Process Steps

Step 1: Tool Initialization

```
# Initialize data pools
self.first_names = ["John", "Jane", "Michael", ...]
self.last_names = ["Smith", "Johnson", "Williams", ...]
self.domains = ["gmail.com", "yahoo.com", ...]
self.cities = ["New York", "Los Angeles", ...]
self.states = ["AL", "AK", "AZ", ...]
self.countries = ["United States", "Canada", ...]
```

Step 2: Scenario Distribution Planning

```
scenario_distribution = {
    "Valid Sign Up": 100,      # 20% - Happy path testing
    "Valid Sign In": 50,       # 10% - Login functionality
    "Empty Field": 75,         # 15% - Required field validation
    "Invalid Format": 75,      # 15% - Format validation
```

```
    "Password Testing": 50,    # 10% - Password validation
    "Boundary Value": 25,      # 5% - Boundary testing
    "Special Characters": 25,  # 5% - Character handling
    "Unicode Characters": 25,  # 5% - Internationalization
    "Security Testing": 30,    # 6% - Security validation
    "Data Quality": 45         # 9% - Data integrity
}
```

Step 3: Data Generation Execution

```
# Generate rows for each scenario
for scenario, count in scenario_distribution.items():
    for _ in range(count):
        field_name = self.generate_field_name()
        test_data = self.generate_test_data(field_name, scenario)
        # Create row with metadata and add to results
```

Step 4: Output Generation

```
# Write to CSV file with proper formatting
with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(['Test Data ID', 'Scenario', 'Field Name', 'Test
Data',
                     'Data Type', 'Validation Rule', 'Expected Result',
'Notes'])
    writer.writerows(rows)
```

6. Explanation of Source Code (Custom Tool)

6.1 Tool Architecture Overview

The custom data generation tool is built using object-oriented programming in Python with the following key components:

Main Class Structure:

```
class TestDataGenerator:
    def __init__(self):
        # Initialize data pools and configuration
        self.test_data_id = 1
        self.first_names = [...]
        self.last_names = [...]
        # ... more data pools

    def generate_500_rows(self):
        # Main method to generate test data
```

```
    def generate_valid_data(self):
        # Generate valid test data

    def generate_invalid_data(self):
        # Generate invalid test data for negative testing
```

6.2 Key Methods and Algorithms

Email Generation Algorithm:

```
def generate_valid_email(self):
    """Generate realistic email addresses"""
    first_name = random.choice(self.first_names).lower()
    last_name = random.choice(self.last_names).lower()
    domain = random.choice(self.domains)
    number = random.randint(1, 999)
    return f"{first_name}.{last_name}{number}@{domain}"
```

Password Generation Algorithm:

```
def generate_valid_password(self):
    """Generate valid passwords with random length"""
    length = random.randint(9, 40)
    chars = string.ascii_letters + string.digits + "!@#$%^&*"
    return ''.join(random.choice(chars) for _ in range(length))
```

Date Generation Algorithm:

```
def generate_valid_dob(self):
    """Generate valid date of birth"""
    start_date = datetime(1950, 1, 1)
    end_date = datetime(2005, 12, 31)
    time_between_dates = end_date - start_date
    days_between_dates = time_between_dates.days
    random_number_of_days = random.randrange(days_between_dates)
    random_date = start_date + timedelta(days=random_number_of_days)
    return random_date.strftime("%Y-%m-%d")
```

Invalid Data Generation:

```
def generate_invalid_data(self, field_name):
    """Generate invalid data based on field type"""
    if field_name == "first_name":
        invalid_types = [
```

```
            "",  # Empty
            "John123",  # Contains numbers
            "John-Doe",  # Contains special chars
            "José",  # Unicode
            "<script>alert('xss')</script>",  # XSS
            "' OR '1'='1",  # SQL injection
            # ... more invalid types
        ]
        return random.choice(invalid_types)
```

6.3 Data Pool Management

Realistic Data Pools:

```
# Names pool for realistic user data
self.first_names = [
    "John", "Jane", "Michael", "Sarah", "David", "Emily", "Robert",
"Lisa",
    "James", "Jennifer", "William", "Jessica", "Richard", "Amanda",
"Thomas",
    # ... 40+ realistic names
]

# Geographic data pools
self.cities = [
    "New York", "Los Angeles", "Chicago", "Houston", "Phoenix",
"Philadelphia",
    "San Antonio", "San Diego", "Dallas", "San Jose", "Austin",
"Jacksonville",
    # ... 29 major US cities
]

self.states = [
    "AL", "AK", "AZ", "AR", "CA", "CO", "CT", "DE", "FL", "GA", "HI",
"ID",
    # ... all 50 US state abbreviations
]
```

6.4 Error Handling and Data Validation

Null Value Handling:

```
# Handle None values for CSV
if test_data is None:
    test_data = "null"
```

Encoding and File I/O:

```
# Write to CSV file with proper encoding
with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
    writer = csv.writer(csvfile)
    # ... write data
```

7. Sample Data

7.1 Valid Data Samples

```
TD001,Valid Sign Up,state,IA,String,Required,Valid,Standard valid input
TD008,Valid Sign Up,first_name,Elizabeth,String,Required + Letters
only,Valid,Standard valid input
TD022,Valid Sign
Up,email,elizabeth.allen870@protonmail.com,String,Required + Email
format,Valid,Standard valid email
TD033,Valid Sign Up,password,R6F*IQdyGdUVWlXp5,String,Required + 9-40
chars,Valid,Standard valid input
TD045,Valid Sign Up,password,dSOeJ2cj6SljBn%yWUeK6Z8KzS^c,String,Required
+ 9-40 chars,Valid,Standard valid input
TD067,Valid Sign Up,dob,2001-07-23,String,Required,Valid,Standard valid
input
TD091,Valid Sign Up,phone,7324359739,String,Required + Numbers
only,Valid,Standard valid input
TD101,Valid Sign In,email,paul.gonzalez255@mail.com,String,Required +
Email format,Valid,Standard valid input
TD103,Valid Sign In,password,SiRQ8@r5Qoty,String,Required + 9-40
chars,Valid,Standard valid input
```

7.2 Invalid Data Samples

```
TD151,Empty Field,password,12345678,String,Required + 9-40
chars,Invalid,Empty required field
TD185,Empty Field,first_name,<script>alert('xss')</script>,String,Required
+ Letters only,Invalid,Empty required field
TD193,Empty Field,email,<script>alert('xss')
</script>@example.com,String,Required + Email format,Invalid,Empty
required field
TD194,Empty Field,password,12345678,String,Required + 9-40
chars,Invalid,Empty required field
TD195,Empty Field,address,null,String,Required,Invalid,Empty required
field
```

7.3 Security Test Data Samples

```
TD185,Empty Field,first_name,<script>alert('xss')</script>,String,Required
+ Letters only,Invalid,Empty required field
TD193,Empty Field,email,<script>alert('xss')
</script>@example.com,String,Required + Email format,Invalid,Empty
required field
TD195,Empty Field,address,null,String,Required,Invalid,Empty required
field
```

7.4 Boundary Value Samples

```
TD025,Boundary Value,password,12345678,String,9-40 chars,Invalid,Exactly
at minimum-1
TD026,Boundary Value,password,123456789,String,9-40 chars,Valid,Exactly at
minimum
TD027,Boundary
Value,password,1234567890123456789012345678901234567890,String,9-40
chars,Valid,Exactly at maximum
TD028,Boundary
Value,password,12345678901234567890123456789012345678901,String,9-40
chars,Invalid,Exactly at maximum+1
```

8. Scenario Testing Framework

8.1 Scenario Categories

1. **Happy Path Scenarios (SC001-SC002)**
    - Successful user registration
    - Successful user login
2. **Negative Testing Scenarios (SC003-SC011)**
    - Validation error handling
    - Invalid input processing
    - Duplicate data handling
3. **Security Testing Scenarios (SC012-SC014)**
    - Account lockout mechanisms
    - SQL injection prevention
    - XSS attack prevention
4. **Boundary Testing Scenarios (SC015)**
    - Password length boundaries
    - Input length limits
5. **Data Quality Scenarios (SC016-SC020)**

    - Special character handling
    - Unicode support
    - Whitespace processing
    - Null value handling

6. **Performance & Integration Scenarios (SC021-SC025)**
    - Concurrent user handling
    - Session management
    - Email verification flows
    - Password reset functionality

8.2 Scenario Execution Plan

Phase 1: Core Functionality (Week 1)

- Execute happy path scenarios (SC001-SC002)
- Test basic validation (SC003-SC010)
- Verify data persistence

Phase 2: Security Testing (Week 2)

- Execute security scenarios (SC012-SC014)
- Perform penetration testing
- Validate security measures

Phase 3: Edge Cases (Week 3)

- Execute boundary testing (SC015)
- Test data quality scenarios (SC016-SC020)
- Validate error handling

Phase 4: Integration Testing (Week 4)

- Execute performance scenarios (SC021)
- Test session management (SC022)
- Validate email flows (SC024-SC025)

9. Quality Assurance and Validation

9.1 Data Quality Checks

- **Uniqueness:** Each test data ID is unique (TD001-TD525)
- **Completeness:** All required fields are covered
- **Realism:** Data follows real-world patterns
- **Coverage:** All validation rules are tested

9.2 Distribution Validation

- **Scenario Distribution:** Proper distribution across 16 scenarios
- **Field Distribution:** Balanced coverage of all 11 fields

- **Data Types:** Appropriate data types for each field
- **Validation Rules:** All rules properly represented

10. Use of AI Tools

10.1 AI Tools Utilized

Primary AI Assistant: Cursor AI (Claude Sonnet 4)

- **Tool Name:** Cursor AI with Claude Sonnet 4 model
- **Purpose:** Test case generation, code analysis, and documentation assistance

10.2 AI-Generated Components

Fully AI-Generated:

- **Test Case Structure**: Complete framework for 25 detailed test scenarios
- **Scenario Categories**: Classification into Happy Path, Negative, Security, Boundary, Data Quality, and Performance
- **Test Steps**: Detailed step-by-step instructions for each scenario
- **Expected Results**: Comprehensive expected outcomes for all test cases
- **Bug Analysis**: Source code analysis to identify actual bugs and vulnerabilities
- **Bug Report**: Complete bug report with 13 identified issues

AI-Assisted Manual Creation:

- **Test Data Mapping**: Framework provided by AI, specific data references refined manually
- **Priority Assignment**: Initial priorities suggested by AI, final assignments based on business impact
- **Category Classification**: Basic structure from AI, specific categorizations refined manually
- **Actual Results**: AI analyzed source code, manual verification of findings

10.3 Prompts Used

Primary Prompts:

1. *"Create comprehensive test scenarios for sign in and sign up features covering all testing aspects"*
2. *"Generate detailed test cases with steps, expected results, and test data references"*
3. *"Analyze the source code to identify actual bugs and fill in actual results and status"*
4. *"Create a bug report CSV file with all identified issues including priority and severity"*

Refinement Prompts:

1. *"Add more security test scenarios including SQL injection and XSS testing"*
2. *"Include boundary value testing for all input fields"*
3. *"Add data quality scenarios for special characters and unicode"*
4. *"Create performance and integration test scenarios"*
5. *"Analyze frontend and backend code for validation rules and security vulnerabilities"*

10.4 Validation and Refinement Process

Test Case Validation:

- **Manual Review**: All generated test cases were reviewed for completeness and accuracy
- **Business Logic**: Verified test scenarios align with application requirements
- **Coverage Analysis**: Ensured comprehensive coverage of all features and edge cases
- **Data Mapping**: Validated test data references and scenario distribution

Source Code Analysis:

- **Frontend Analysis**: Manually verified AI's analysis of validation rules and UX issues
- **Backend Analysis**: Confirmed AI's identification of security vulnerabilities
- **Bug Verification**: Cross-referenced AI findings with actual code behavior
- **Priority Assessment**: Manually reviewed and adjusted bug priorities and severities

Documentation Enhancement:

- **Manual Review**: All AI-generated documentation was reviewed for accuracy
- **Clarity Improvements**: Enhanced explanations and added missing details
- **Structure Optimization**: Reorganized content for better readability
- **Example Addition**: Added specific examples and practical scenarios

10.5 AI vs Manual Creation Breakdown

Component	AI Contribution	Manual Refinement
Test Case Framework	100%	Business logic verification
Scenario Categories	90%	Specific categorizations
Test Steps	100%	Clarity and completeness
Expected Results	100%	Accuracy verification
Source Code Analysis	80%	Manual verification
Bug Identification	90%	Priority and severity assessment
Bug Report Structure	100%	Content review and enhancement
Test Data Mapping	70%	Specific data references

10.6 Quality Assurance Measures

AI Output Validation:

- **Completeness Check**: Verified all scenarios cover required testing aspects
- **Logic Verification**: Tested scenario logic for correctness and completeness
- **Coverage Analysis**: Ensured comprehensive test scenario coverage
- **Source Code Verification**: Manually verified AI's code analysis findings

Manual Enhancement:

- **Business Requirements**: Aligned test cases with specific business needs

- **Edge Cases**: Added additional boundary and error scenarios
- **Documentation**: Enhanced explanations and added practical examples
- **Bug Assessment**: Refined priorities and severities based on business impact

11. Conclusion

The scenario testing framework provides comprehensive coverage for the sign in and sign up features with:

- **525 test data rows** generated (exceeds required 500)
- **25 detailed scenarios** covering all testing aspects
- **16 different scenario types** with proper distribution
- **11 input fields** with complete coverage
- **Custom data generation tool** for efficient testing
- **Realistic test data** following real-world patterns
- **Security testing** data for vulnerability assessment

The combination of comprehensive scenarios and automated data generation provides a solid foundation for systematic testing of the authentication features.

11. Appendices

11.1 Tool Files

- `data_generator.py` - Main Python script (435 lines)
- `run_generator.bat` - Windows execution script
- `run_generator.sh` - macOS/Linux execution script
- `README_Data_Generator.md` - Complete documentation

11.2 Generated Output

- `500_test_data_rows.csv` - 525 test data rows
- Console output with statistics and distribution

11.3 Scenario Files

- `scenario-testing-signin-signup.csv` - 25 detailed test scenarios
- `test-data-sets-signin-signup.csv` - 15 organized data sets

Report Generated: December 2024
Scenario Testing Framework: Comprehensive 25-scenario approach
Data Generation Tool: Custom Python script
Application: sprint5-with-bugs
Features: Sign In and Sign Up
Total Test Data Generated: 525 rows